arXiv:2609.23896v2 [cs.RO] 30 Oct 2026

BarrierFormer: Transformer-Guided Predictive Barrier Enforcement for Safe Robot Control

Anandsingh Chauhan
Arizona State University
achauh67@asu.edu

Kunal Garg
Arizona State University
kgarg24@asu.edu

Abstract: Control barrier functions (CBFs) have become one of the most popular tools for encoding and enforcing state constraints in safety-critical robotics. Standard CBF approaches are inherently myopic in nature as they enforce safety only at the current time step. Consequently, the system can be driven toward the boundary of the safe set where no feasible safe control exists at a future timestep. Model predictive control (MPC) based approaches address this by enforcing state constraints over a receding horizon. However, such approaches generally require the model to be known for solving a constrained optimization problem at every step, which is computationally expensive for real-time deployment. We propose BarrierFormer, a barrier-supervised transformer framework that addresses these limitations by encoding rollout-level CBF constraints in learning a model-free safe policy. A causal transformer encodes observation-action history, autoregressively generates a predictive rollout through the dynamics head to replace the model, and provides a residual correction to a nominal controller through the action head to replace the online computation. A barrier critic operating on local observations evaluates CBF constraint violations along this rollout, and a safety teacher computes barrier-consistent actions satisfying these constraints as direct supervision targets for the learned control policy. During inference, the policy maps observation-action history to control actions without any online optimization or model knowledge, enabling real-time model-free predictive safety enforcement. Evaluations across linear and nonlinear, 2D and 3D dynamical systems for safe goal-directed navigation demonstrate that BarrierFormer outperforms existing reinforcement learning (RL)-based, diffusion-based, MPC-based, and transformer-based approaches in safety rate and inference latency.[1]

Keywords: Control Barrier Function, Predictive Control, Causal transformers

1 Introduction

Safe robot navigation requires an agent to reach a goal while avoiding obstacles [1, 2, 3]. This reach-avoid objective couples two aspects: the controller must make progress toward the goal while avoiding actions that lead to future safety violations [4]. Control barrier functions (CBFs) provide a principled framework for the safety component by converting set-invariance requirements into constraints on the control input [5, 6, 7], minimally modifying a nominal controller to be safe. An online control input is computed using a single-step quadratic program (QP) to enforce these constraints, but this pointwise formulation is myopic: the barrier condition is evaluated only at the current step and does not constrain how the closed-loop trajectory evolves [8, 9, 10]. An action may therefore be instantaneously safe while still driving the system toward states where the barrier constraint becomes infeasible and no safe recovery action remains [8, 11]. Model predictive control (MPC) and discrete-time CBF-MPC mitigate this by enforcing safety over predicted trajectories [11,

[1] Project website: https://safe-robotics-group-at-asu.github.io/BarrierFormer-website/

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

12], but require model knowledge for solving constrained optimization problems online, making real-time deployment computationally expensive [13, 14].

The challenge is further compounded when the agent operates under partial observations. The agent observes only its own state and LiDAR readings within a fixed sensing radius, with obstacle geometry beyond that range entirely unknown at decision time [15, 16, 17]. A single observation is therefore insufficient to characterize the safety of the current state: the agent may be *currently* safe but on a trajectory toward an obstacle that has not yet entered its sensing range. Certifying safety, therefore, requires generating a predictive rollout from the current state to anticipate future constraint violations. Diffusion-based approaches incorporate safety through predefined CBFs via iterative optimization at inference time [18, 19]. Transformer-based policies are better suited for this setting, leveraging self-attention to aggregate temporal information across observation-action histories [20, 21]; jointly predicting actions and future states through a shared transformer further improves policy performance [22]. However, incorporating safety filters or constrained optimization at deployment [21, 23, 24] sacrifices inference-time efficiency. The key question is therefore whether safety constraints evaluated over predicted rollouts can be encoded directly into policy parameters, such that no online optimization is needed at deployment.

In this work, we use a barrier-supervised transformer framework that directly encodes rollout-level CBF constraints into policy parameters via training-time supervision. A causal transformer encodes observation-action history and jointly outputs a residual correction to a nominal controller and predicted state increments, which are used to autoregressively generate a predictive rollout (see Fig. 1). A barrier critic evaluates CBF constraint violations along this rollout, and a safety teacher computes barrier-consistent corrections as direct supervision targets for the policy. Since horizon-level safety reasoning is encoded in the policy parameters via training-time barrier supervision, the policy maps observation-action histories directly to control actions, with no online optimization at inference.

The contributions of this work are as follows. First, we propose BarrierFormer, a model-free framework for safety enforcement that encodes observation-action histories and incorporates barrier constraints over a future horizon into safe policy learning. Second, we replace the online computation of the safe input over a horizon with a learned policy, leading to a faster runtime inference compared to existing predictive methods [12, 18]. Third, BarrierFormer uses past observation-action history, where observation includes the robot state and local LiDAR measurements, without access to a global map, to predict state increments and reconstruct future observations, thereby enforcing safety with partial environmental information. We evaluate BarrierFormer on linear and nonlinear robot dynamics against RL-based, diffusion-based, MPC-based, and transformer-based baselines, demonstrating improved safety rate, faster inference, and zero-shot generalization to larger and more cluttered environments.

2 Related Work

Optimization-based. CBF-QP controllers enforce safety by solving a quadratic program at each timestep that minimally modifies a nominal control input to satisfy the barrier condition [5, 6]. While real-time efficient, this formulation is myopic and can drive the system toward states where the barrier constraint becomes infeasible [8]. Discrete-time CBF-MPC methods address this by enforcing barrier constraints over a receding horizon [11, 12], but the resulting optimization is generally nonconvex and requires accurate dynamics models and full state estimates, making real-time deployment expensive.

Learning-based. CBFs have been integrated with RL to improve constraint satisfaction through action projection or rollout filtering during training [25, 26, 27]. These methods often require a safety filter or action projection at inference to improve safety, instead of using the policy output directly. Other approaches learn safety certificates directly: neural CBF methods construct value-function-based safety filters [24], and BarrierNet embeds differentiable quadratic programs into neural controllers [23]. Both enable end-to-end training of safety-aware policies but retain a safety filter or constrained optimization at inference.

Generative Models for Robot Control. Diffusion-based planners incorporate safety by embedding predefined CBFs into the denoising process via gradient projection [18, 19]. While these methods improve constraint satisfaction at the trajectory level, they rely on iterative denoising during inference and are not well-suited for real-time control. Transformer-based policies leverage self-attention to aggregate temporal information across observation-action histories for history-conditioned control [20, 21, 28]. PACT does not enforce safety constraints [20], while TransformerMPC retains online optimization at inference [28]. ConBaT [21] augments a causal transformer with a learned barrier critic trained on safe and unsafe demonstrations, and at inference applies a lightweight gradient-based optimization to correct the proposed action only when the critic predicts the next state to be unsafe, rather than encoding safety directly into the policy parameters.

3 Preliminaries and Problem Formulation

Notations: Let $\mathbb{R}$ and $\mathbb{Z}_{\geq 0}$ denote the sets of real numbers and nonnegative integers, respectively. We use $\|\cdot\|$ to denote the Euclidean norm and $[\cdot]_+$ to denote $\max(0, \cdot)$.

System and Observation Model: We consider the reach-avoid problem, in which an autonomous agent safely navigates an obstacle-filled environment to reach a goal position $\mathbf{p}_{\text{goal}} \in \mathbb{R}^{n_p}$ [1]. Let $\mathbf{x}_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ and $\mathbf{u}_t \in \mathcal{U} \subset \mathbb{R}^{n_u}$ denote the state and control respectively at time $t \in \mathbb{Z}_{\geq 0}$, where $\mathcal{U}$ is a compact admissible control set. Each state $\mathbf{x}_t \in \mathcal{X}$ includes a position component $\mathbf{p}_t \in \mathbb{R}^{n_p}$, where $n_p \in \{2, 3\}$ denotes the dimension of the position vector. The agent dynamics are given by:

$$\mathbf{x}_{t+1} = F(\mathbf{x}_t, \mathbf{u}_t) \coloneqq f(\mathbf{x}_t) + g(\mathbf{x}_t)\mathbf{u}_t. \tag{1}$$

The agent has no prior knowledge of obstacle locations and perceives its environment through n_{rays} evenly spaced LiDAR rays within a fixed sensing radius $R > 0$. Let $o : \mathcal{X} \to \mathbb{R}^{n_o}$ denote the observation map. The observation at time t is

$$\mathbf{o}_t = o(\mathbf{x}_t) = \left(\mathbf{x}_t,\ \mathbf{p}_{\text{goal}} - \mathbf{p}_t,\ \mathbf{y}_{t,1}, \ldots, \mathbf{y}_{t,n_{\text{rays}}}\right) \in \mathbb{R}^{n_o}. \tag{2}$$

We drop the time index t and the state argument $\mathbf{x}$ when clear from context. For each $j \in \{1, \ldots, n_{\text{rays}}\}$, $\mathbf{y}_{t,j} \in \mathbb{R}^{n_p+2}$ denotes the feature vector of the j-th LiDAR ray: $\mathbf{y}_{t,j} = \left(m_{t,j}, \bar{d}_{t,j}, \mathbf{v}_{t,j}\right)$, where $m_{t,j}$ is the hit indicator, $\bar{d}_{t,j}$ denotes the return distance of the j-th LiDAR ray, normalized by the sensing radius R, and $\mathbf{v}_{t,j} \in \mathbb{R}^{n_p}$ is the ray direction.

Safety Specification: Let $r \in (0, R)$ denote the agent radius and $\mathcal{O} = \{\mathcal{O}_1, \ldots, , \mathcal{O}_M\}$ the set of obstacles, with $\mathcal{O}_i \subset \mathbb{R}^{n_p}$ for each $i \in \{1, \ldots, M\}$. The prescribed safe set $\mathcal{S} \subset \mathcal{X}$ and avoid set $\mathcal{A} \subset \mathcal{X}$ are defined as:

$$\mathcal{S} = \{\mathbf{x} \in \mathcal{X} \mid d(\mathbf{p}, \mathcal{O}_i) \geq r,\ \forall i \in \{1, \ldots, M\}\}, \qquad \mathcal{A} = \mathcal{X} \setminus \mathcal{S}. \tag{3}$$

where $\mathbf{p} \in \mathbb{R}^{n_p}$ is the position component of $\mathbf{x}$, and $d(\mathbf{p}, \mathcal{O}_i) = \min_{\mathbf{y} \in \mathcal{O}_i} \|\mathbf{p} - \mathbf{y}\|$ is the distance from $\mathbf{p}$ to the obstacle $\mathcal{O}_i$. The *safety* requirement is that the system trajectories remain in the set $\mathcal{S}$ at all times, i.e., $\mathbf{x}_t \in \mathcal{S}$ for all t. A set $\mathcal{C} \subseteq \mathcal{X}$ is *control invariant* for the system in (1) if, for every $\mathbf{x} \in \mathcal{C}$, there exists $\mathbf{u} \in \mathcal{U}$ such that $F(\mathbf{x}, \mathbf{u}) \in \mathcal{C}$ [29]. A state-feedback policy $\pi : \mathcal{X} \to \mathcal{U}$ renders $\mathcal{C}$ *forward invariant* if $\mathbf{x}_0 \in \mathcal{C}$ implies $\mathbf{x}_t \in \mathcal{C}$ for all $t \geq 0$ under $\mathbf{x}_{t+1} = F(\mathbf{x}_t, \pi(\mathbf{x}_t))$. If $\mathcal{C} \subseteq \mathcal{S}$, forward invariance of $\mathcal{C}$ implies system safety with respect to $\mathcal{S}$. A barrier function, defined below, certifies control invariance of a subset of $\mathcal{S}$ [5, 6, 30].

Definition 3.1 (Discrete-Time Control Barrier Function [11, 31]) *A function $h : \mathbb{R}^{n_o} \to \mathbb{R}$ is a discrete-time control barrier function (DTCBF) for the system in (1) with observation model (2) on $\mathcal{C}_h := \{\mathbf{x} \in \mathcal{X} \mid h(o(\mathbf{x})) \geq 0\} \subseteq \mathcal{S}$ if there exists a continuous function $\alpha : \mathbb{R} \to \mathbb{R}$ that is strictly increasing with $\alpha(0) = 0$ and $0 < \alpha(s) \leq s$ for all $s > 0$, such that for every $\mathbf{x} \in \mathcal{C}_h$ there exists $\mathbf{u} \in \mathcal{U}$ such that*

$$h(\mathbf{o}^+) - h(\mathbf{o}) \geq -\alpha(h(\mathbf{o})), \tag{4}$$

where $\mathbf{o} := o(\mathbf{x})$ and $\mathbf{o}^+ := o(F(\mathbf{x}, \mathbf{u}))$.

We use the linear function $\alpha(s) = \gamma s$ with $\gamma \in (0, 1]$, yielding $h(\mathbf{o}^+) \geq (1-\gamma)h(\mathbf{o})$ from (4). Based on the DTCBF definition, the set of barrier-admissible control inputs is defined as:

$$\mathcal{K}_h(\mathbf{x}) := \{\mathbf{u} \in \mathcal{U} \mid h(o(F(\mathbf{x}, \mathbf{u})) - h(o(\mathbf{x})) + \alpha(h(o(\mathbf{x}))) \geq 0\}. \tag{5}$$

Any observation-feedback policy $\pi : \mathbb{R}^{n_o} \to \mathcal{U}$ satisfying $\pi(o(\mathbf{x})) \in \mathcal{K}_h(\mathbf{x})$, for all $\mathbf{x} \in \mathcal{C}_h$, renders $\mathcal{C}_h$ forward invariant [11, 31].

Remark 3.1 (Myopia of Pointwise DTCBF Enforcement) *Condition (4) constrains a single transition at a given time-step t, and does not guarantee that the system can remain safe in the future. Due to this myopic nature of the CBF, a control input satisfying (4) at every timestep can still drive the system towards a state $\mathbf{x}$ where $\mathcal{K}_h(\mathbf{x})$ is empty [8, 11], making the barrier constraint infeasible. This motivates enforcing the barrier condition over a predicted rollout rather than pointwise at the current state. Similar ideas have been explored previously by combining the CBF-based approaches with MPC-like lookahead mechanisms [12, 32, 33, 34].*

The myopic nature of the CBF constraint in (4) can be overcome by enforcing the CBF constraint over a horizon $\{t+k\}_{k=0}^{H}$ for some H at each time step t as:

$$h(\mathbf{o}_{t+k+1}) \geq (1-\gamma)\, h(\mathbf{o}_{t+k}), \quad k = 0, \ldots, H-1, \tag{6}$$

which requires *computation* of future states per (1), which in turn requires a known system model. In a model-free control design, future states must be *predicted* from available information, such as historical state-action pairs. Based on this, we formulate the problem considered in this work.

Problem 3.1 *Design a model-free, history-conditioned policy π_θ such that the closed-loop trajectories of (1) satisfy **safety**, i.e., the robot does not collide with any obstacle, i.e., $d(\mathbf{p}_t, \mathcal{O}_i) \geq r$, $\forall i \in \{1, \ldots, M\}$, $\forall t \geq 0$, where safety at each time step is enforced by requiring the barrier condition to hold over the predicted rollout (6); and **liveness**, i.e., $\inf_{t \geq 0} \|\mathbf{p}_t - \mathbf{p}_{\text{goal}}\| = 0$.*

4 Methodology

BarrierFormer overview: In the model-free, partially observed setting, generating a predicted rollout requires using previous observations and actions to infer how the robot state evolves. We therefore use a GPT-2-style causal transformer [35], which conditions residual-action and state-increment predictions on the preceding finite observation–action history containing T observations and the $T-1$ preceding actions, where T is a positive integer:

$$\mathcal{H}_t := (\mathbf{o}_{t-T+1}, \mathbf{u}_{t-T+1}, \ldots, \mathbf{o}_{t-1}, \mathbf{u}_{t-1}, \mathbf{o}_t) \in \mathbb{H}_T. \tag{7}$$

At each rollout step t, the predicted state increment d_{θ_x} is used to obtain the next state $\hat{\mathbf{x}}_{t+1}$, from which the next observation $\hat{\mathbf{o}}_{t+1}$ is constructed. The current rollout history $\mathcal{H}_t$ is advanced to $\mathcal{H}_{t+1}$ by discarding the oldest observation-action pair and appending the predicted action and reconstructed observation. During training, BarrierFormer uses these predictions to enforce the barrier condition over a future horizon $t, t+1, \ldots, t+H$, instead of only at the current time step t, as discussed in Remark 3.1. The proposed BarrierFormer operates in two steps, as illustrated in Fig. 1. In Step 1, a causal transformer encodes the observation-action history $\mathcal{H}_t$ and jointly predicts a residual correction to the action produced by a nominal control policy via the actor head and state increments via the dynamics head. These predictions are used to autoregressively generate a finite-horizon rollout. In Step 2, a barrier critic evaluates DTCBF constraint violations along the predicted rollout, and an SQP-based safety teacher computes barrier-consistent corrections as direct supervision targets for the actor head. Step 2 operates only during training; at deployment, the transformer maps the observation-action history to control actions directly, with horizon-level safety reasoning encoded into the policy parameters through training-time barrier supervision, thereby removing the requirement of any online constrained optimization.

Each observation and action in $\mathcal{H}_t$ is independently mapped to a d_h-dimensional embedding using separate linear encoders. The resulting $2T-1$ embeddings are interleaved in temporal order to

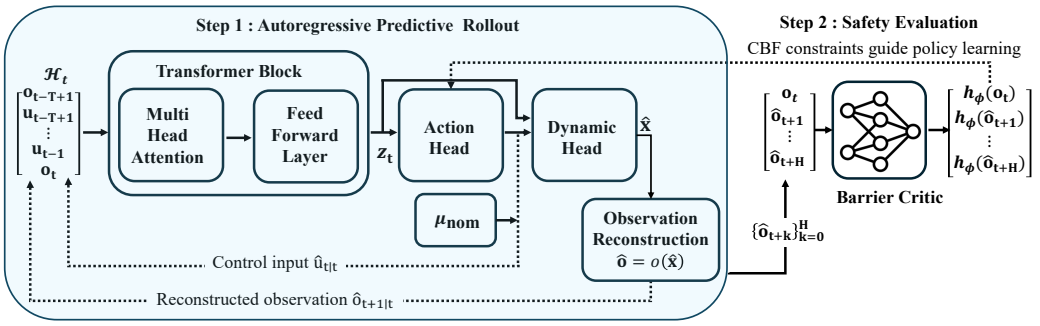


Figure 1: Overview of BarrierFormer: In Step 1, a causal transformer autoregressively generates a finite-horizon predictive rollout via the actor and dynamics heads. In Step 2, a barrier critic h_ϕ evaluates DTCBF constraint violations along the predicted observations.

form the input token sequence. The backbone transformer T_{θ_T}, parameterized by θ_T, processes this sequence and produces a latent representation $\mathbf{z}_t = T_{\theta_T}(\mathcal{H}_t)$ that is shared between two heads. The first one is *action head* π_{θ_u}, an MLP that maps $\mathbf{z}_t$ to a residual correction $\Delta\mathbf{u}_t = \pi_{\theta_u}(\mathbf{z}_t)$, where the deployed control input is $\mathbf{u}_t = \mu_{\text{nom}}(\mathbf{x}_t) + \Delta\mathbf{u}_t$, with μ_{nom} being a nominal control policy designed to drive the robot toward the goal, but not necessarily to satisfy the safety constraints. The second one is *dynamics head* d_{θ_x}, an MLP that maps $(\mathbf{z}_t, \mathbf{u}_t)$ to a predicted state increment $\Delta\hat{\mathbf{x}}_t = d_{\theta_x}(\mathbf{z}_t, \mathbf{u}_t)$, used during training to generate predictive rollouts. In particular, with $\hat{\mathbf{x}}_{t|t} = \mathbf{x}_t$ and $\widehat{\mathcal{H}}_{t|t} = \mathcal{H}_t$, at each step $k = 0, \ldots, H-1$, BarrierFormer computes:

$$\mathbf{z}_{t+k|t} = T_{\theta_T}(\mathcal{H}_{t+k|t}), \tag{8}$$

$$\hat{\mathbf{u}}_{t+k|t} = \mu_{\text{nom}}(\hat{\mathbf{x}}_{t+k|t}) + \pi_{\theta_u}(\mathbf{z}_{t+k|t}), \tag{9}$$

$$\hat{\mathbf{x}}_{t+k+1|t} = \hat{\mathbf{x}}_{t+k|t} + d_{\theta_x}(\mathbf{z}_{t+k|t}, \hat{\mathbf{u}}_{t+k|t}). \tag{10}$$

At each step k, the predicted observation $\hat{\mathbf{o}}_{t+k+1|t}$ is reconstructed from $\hat{\mathbf{x}}_{t+k+1|t}$ using the observation structure in (2): the robot state and goal-relative position are already known to the robot, while the predicted LiDAR rays are generated using the historical data of the environment, i.e., obstacles encountered thus far. At each rollout step, the predicted history $\widehat{\mathcal{H}}_{t+k|t}$ is updated by discarding its oldest observation-action pair and appending $(\hat{\mathbf{u}}_{t+k|t}, \hat{\mathbf{o}}_{t+k+1|t})$, yielding $\widehat{\mathcal{H}}_{t+k+1|t}$. The resulting sequence of predicted observations $\{\hat{\mathbf{o}}_{t+k|t}\}_{k=0}^{H}$ is passed to the barrier critic for safety evaluation.

Barrier critic: Given the predicted observation sequence $\{\hat{\mathbf{o}}_{t+k|t}\}_{k=0}^{H}$, a barrier critic $h_\phi : \mathbb{R}^{n_o} \to \mathbb{R}$ evaluates horizon-level safety by mapping each predicted observation to a scalar barrier value. Operating on local observations rather than the full environmental state makes the critic directly applicable to partially observable settings. At time t, safety is evaluated by checking whether the DTCBF condition is satisfied along the predicted observation rollout:

$$h_\phi\left(\hat{\mathbf{o}}_{t+k+1}\right) \geq (1-\gamma)\, h_\phi\left(\hat{\mathbf{o}}_{t+k}\right), \quad k = 0, \ldots, H-1. \tag{11}$$

The per-step violation of (11) is $v_k(\mathbf{o}_t) = \left[\eta_v + (1-\gamma)\, h_\phi(\hat{\mathbf{o}}_{t+k|t}) - h_\phi(\hat{\mathbf{o}}_{t+k+1|t})\right]_+$, where $\eta_v \geq 0$ is a margin tightening the barrier constraint. The horizon-level violation is obtained by approximating the worst-case violation $\max_k v_k(\mathbf{o_t})$ via a smooth log-sum-exp approximation:

$$V(\mathbf{o}_t) = \frac{1}{\beta} \log\left(\frac{1}{H}\sum_{k=0}^{H-1} \exp\left(\beta\, v_k(\mathbf{o}_t)\right)\right), \quad \beta > 0, \tag{12}$$

where $\beta \to \infty$ recovers $\max_k v_k(\mathbf{o}_t)$, and the normalization by H ensures $V(\mathbf{o}_t) = 0$ when all constraints are satisfied.

SQP-based safety teacher: The safety teacher in Step 2 (see Fig. 1) operates only during training to generate corrective-action labels for the actor head. Given the horizon barrier constraints (11) evaluated by the barrier critic h_ϕ, the goal is to find the sequence of residual corrections $\Delta\mathbf{U}_t = (\Delta\mathbf{u}_{t|t}, \ldots, \Delta\mathbf{u}_{t+H-1|t})$ that minimally modifies the nominal controller while satisfying all horizon barrier constraints. Although SQP is used only during training to generate supervision for the

actor head, its candidate trajectories are propagated using the simulator dynamics available during training, $\mathbf{x}_{t+k+1|t} = F(\mathbf{x}_{t+k|t}, \mathbf{u}_{t+k|t})$. This avoids training the actor head on corrective labels generated under a different rollout distribution. At deployment, the SQP teacher is not used; the actor head predicts the residual correction directly from the observation-action history $\mathcal{H}_t$. The CBF constraints are then enforced on the resulting predicted observations $\hat{\mathbf{o}}_{t+k|t}$, leading to the following receding horizon optimization problem:

$$\min_{\Delta \mathbf{U}_t} \sum_{k=0}^{H-1} \left\| \mathbf{u}_{t+k|t} - \mu_{\text{nom}}(\mathbf{x}_{t+k|t}) \right\|^2 \quad \text{s.t.} \quad \mathbf{u}_{t+k|t} \in \mathcal{U},\ c_k(\Delta \mathbf{U}_t) \le 0,\ k = 0, \dots, H-1, \tag{13}$$

where $c_k(\Delta \mathbf{U}_t) = \eta_s + (1-\gamma)\, h_\phi(\hat{\mathbf{o}}_{t+k|t}) - h_\phi(\hat{\mathbf{o}}_{t+k+1|t})$ encodes the DTCBF constraint (11) at step k, where $\eta_s \ge 0$ is a safety margin that tightens the barrier constraint to account for the mismatch in the ground truth observations ($\mathbf{o}_{t+k|t}$) and predicted observations ($\hat{\mathbf{o}}_{t+k|t}$). Problem (13) is nonconvex since c_k is a nonconvex function of $\Delta \mathbf{U}_t$ Hence, we solve (13) using sequential quadratic programming (SQP), which iteratively linearizes the nonlinear constraints and solves a sequence of QP subproblems [36]. At each SQP iteration i, the constraints are linearized around the current correction sequence $\Delta \mathbf{U}_t^{(i)}$:

$$c\left(\Delta \mathbf{U}_t^{(i)} + \delta\right) \approx c\Big(\Delta \mathbf{U}_t^{(i)}\Big) + J^{(i)}\delta, \quad J^{(i)} = \left.\frac{\partial c}{\partial \Delta \mathbf{U}_t}\right|_{\Delta \mathbf{U}_t^{(i)}}. \tag{14}$$

A slack variable $\xi \in \mathbb{R}^H$ with quadratic penalty weight $\lambda_\xi > 0$ is introduced to ensure feasibility when the barrier constraints cannot be simultaneously satisfied.

$$\begin{aligned} \min_{\delta,\, \xi} \ & \frac{1}{2} \sum_{k=0}^{H-1} \left\| \Delta \mathbf{u}_{t+k|t}^{(i)} + \delta_k \right\|^2 + \frac{\lambda_\xi}{2} \|\xi\|_2^2 \\ \text{s.t.} \ & c^{(i)} + J^{(i)}\delta \le \xi, \quad \xi \ge 0, \quad \mu_{\text{nom}}\big(\hat{\mathbf{x}}_{t+k|t}^{(i)}\big) + \Delta \mathbf{u}_{t+k|t}^{(i)} + \delta_k \in \mathcal{U}, \quad k = 0, \dots, H-1. \end{aligned} \tag{15}$$

Consistent with the receding-horizon principle, only the first element $\Delta \mathbf{u}_t^\star = [\Delta \mathbf{U}_t^\star]_0$ is retained as the supervision target for the actor head at each training step.

Data collection and labeling: Training data are collected on-policy by periodically executing the learned policy π_θ. Rollout data are stored in a replay buffer as sequences of transitions, while a separate unsafe replay buffer retains rollouts containing at least one collision. During training, a batch of transitions is sampled from these buffers. Each sampled transition contains its observation–action history $\mathcal{H}$, whose final observation is denoted by $\mathbf{o}$. Let $\mathcal{D}_S$ and $\mathcal{D}_U$ denote the sets of observations labelled safe and unsafe, respectively. Although the state component $\mathbf{x}$ of an observation may be collision-free, the trajectory generated by the current policy may still encounter a collision at a later step. We therefore assign labels using the states that follow the observation in the same stored rollout. If $\mathbf{x}$ is in collision, $\mathbf{o}$ is added to $\mathcal{D}_U$. If $\mathbf{x}$ and all subsequent states over the next T_{label} steps are collision-free, $\mathbf{o}$ is added to $\mathcal{D}_S$; otherwise, it remains unlabelled. As $T_{\text{label}} \to \infty$, these labels approach the infinite-horizon safe set under the policy used to generate the rollout, which is intractable to compute [24, 37, 38]. We therefore use $T_{\text{label}} = 32$ in all experiments. The labeling horizon T_{label} is separate from the transformer's predictive horizon H: T_{label} determines how many future states are considered when labeling observations, whereas H determines the horizon over which barrier constraints are evaluated along predicted rollouts.

Loss function design: The transformer backbone T_{θ_T}, actor head π_{θ_u}, dynamics head d_{θ_x}, and barrier critic h_ϕ are trained jointly by minimizing:

$$\min_{\theta_T, \theta_u, \theta_x, \phi} \lambda_{\text{act}} L_{\text{act}}(\theta_T, \theta_u) + \lambda_{\text{dyn}} L_{\text{dyn}}(\theta_T, \theta_x) + L_{\text{bar}}(\theta_T, \theta_u, \phi), \tag{16}$$

where $\lambda_{\text{act}}, \lambda_{\text{dyn}} > 0$ and $\mathbf{z}_t = T_{\theta_T}(\mathcal{H}_t)$ denotes the latent representation.

Actor Loss: The actor loss $L_{\text{act}}(\theta_T, \theta_u) = \sum_{\mathcal{H} \in \mathcal{D}} \|\pi_{\theta_u}(\mathbf{z}_t) - \Delta \mathbf{u}_t^\star\|_2^2$ supervises the residual correction $\Delta \mathbf{u}_t = \pi_{\theta_u}(\mathbf{z}_t)$ using the first action of the safety teacher solution $\Delta \mathbf{u}_t^\star$, computed over all states in $\mathcal{D}$ for which the SQP optimization converges.

Dynamics loss: The dynamics loss supervises the state-increment model d_{θ_x} using observed transitions from the simulator. Let $\tau_t = (\mathcal{H}_t, \mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1})$ denote a transition tuple and define the loss function as $L_{\text{dyn}}(\theta_T, \theta_x) = \sum_{\tau_t \in \mathcal{D}} \|d_{\theta_x}(\mathbf{z}_t, \mathbf{u}_t) - (\mathbf{x}_{t+1} - \mathbf{x}_t)\|_2^2$.

Barrier loss: The barrier loss penalizes DTCBF violations along predicted rollouts and trains h_ϕ to separate safe and unsafe observations. Using the horizon-level violation $V(\mathbf{o})$ defined in (12), the barrier loss combines the rollout loss and classification loss:

$$L_{\text{bar}}(\theta_T, \theta_u, \phi) = \underbrace{\lambda_{\text{bar}} \frac{1}{|\mathcal{D}|} \sum_{\mathbf{o} \in \mathcal{D}} V(\mathbf{o})}_{L_{\text{roll}}} + \underbrace{\frac{\lambda_{\text{safe}}}{|\mathcal{D}_S|} \sum_{\mathbf{o} \in \mathcal{D}_S} [\eta - h_\phi(\mathbf{o})]_+ + \frac{\lambda_{\text{unsafe}}}{|\mathcal{D}_U|} \sum_{\mathbf{o} \in \mathcal{D}_U} [\eta + h_\phi(\mathbf{o})]_+}_{L_{\text{cls}}}. \tag{17}$$

where $\lambda_{\text{bar}} > 0$ weights the rollout loss and $\lambda_{\text{safe}}, \lambda_{\text{unsafe}} > 0$ weight the safe and unsafe classification terms, while $\eta > 0$ defines the classification margin, encouraging $h_\phi(\mathbf{o}) \geq \eta$ for safe observations and $h_\phi(\mathbf{o}) \leq -\eta$ for unsafe observations, preventing the barrier critic from collapsing to a trivial solution near zero. Here, $\mathcal{D}$ denotes the batch of histories sampled from the replay buffers. For each $\mathbf{o} \in \mathcal{D}$, the BarrierFormer generates a predicted rollout, and $V(\mathbf{o})$ evaluates its horizon-level barrier violation according to (12). The sets $\mathcal{D}_S$ and $\mathcal{D}_U$ contain the observations labelled safe and unsafe, respectively. BarrierFormer is trained in two phases. In Phase 1, the transformer backbone T_{θ_T} and dynamics head d_{θ_x} are pretrained using L_{dyn}, while the actor head π_{θ_u} is initialized via behavioral cloning on offline collected trajectories [20, 21, 28, 39], allowing all components to warm-start before online training begins. In Phase 2, the full training objective (16) is minimized with on-policy data collection; the dynamics head d_{θ_x} is updated solely via L_{dyn}, while CBF gradients flow through the barrier critic h_ϕ directly into the actor head π_{θ_u}.

5 Experiments

We evaluate BarrierFormer to answer the following questions: (1) Does it achieve higher safety and success rates than the current state-of-the-art without sacrificing inference latency?; (2) Does training the barrier critic with the learned dynamics head maintain performance compared to training with the true simulator?; and (3) Does it generalize well to out-of-distribution data, e.g., to larger and more cluttered environments?

Environments: We evaluate BarrierFormer on three dynamical systems: one linear (Double Integrator) and two nonlinear (Dubins Car and Crazyflie). The sensing radius is set to $R = 0.5$ for Double Integrator and Dubins Car and $R = 1.0$ for Crazyflie, and the robot radius to $r = 0.05$ across all environments. Training is conducted in a workspace with sidelength $l = 4$ and $M = 8$ obstacles for Double Integrator and Dubins Car, and $l = 3$ and $M = 6$ obstacles for Crazyflie. The predictive horizon, observation-action history length, and labeling horizon are set to $H = 6$, $T = 12$, and $T_{\text{label}} = 32$, respectively. Further details are given in Appendix A.

Evaluation Metrics: We evaluate each method using three metrics. The **safety rate** is defined as the fraction of episodes in which the robot does not collide with any obstacle at any timestep. The **reaching rate** is defined as the fraction of episodes in which the robot reaches its goal location by the end of the episode. The **success rate** is defined as the fraction of episodes in which the robot is both safe and reaches its goal. For each environment, performance is evaluated across 3 random seeds and 32 experiments, reporting mean and standard deviation.

Baselines: We compare BarrierFormer against four baselines based on three different model-free and one model-based paradigms:[2]

RL-based: CBF-RL [25] integrates CBFs into RL training by safety filtering policy rollouts and incorporating a barrier-inspired reward term, but relies on predefined CBFs.

[2] *Implementation Note.* CBF-RL and CoBL-Diffusion were originally evaluated on single integrator systems. To enable fair comparison across all three environments, all baselines are re-implemented within our framework using the same dynamics, observation, and obstacle configurations (refer Appendix).

Diffusion-based: CoBL-Diffusion [19] guides the denoising process of a diffusion model using predefined CBFs via gradient projection, enforcing safety iteratively at inference time.

Transformer-based: ConBaT [21] augments a causal transformer with a learned barrier critic trained on safe and unsafe demonstrations, and at inference applies a lightweight gradient-based optimization to correct the action when the future state is predicted to fall below a safety threshold.

Model-based MPC [40] enforces safety over a receding horizon by solving a constrained optimization problem at each control step, but requires accurate dynamics models and full state estimates, making real-time deployment computationally expensive.

Table 1: Performance comparison in terms of safety rate (Safe), goal reaching rate (Reach), success rate (Success) and per-step computational time (Time) on 2D environments DI: Double Integrator, DC: Dubins Car, and 3D environment CF: Crazyflie drone. †Not reported for CoBL-Diffusion, whose iterative denoising makes inference slow [41].

Env.	Metric	Model-Free				Model-Based	
		CBF-RL	CoBL-Diff	ConBaT	Ours (Dyn-Head)	MPC	Ours (Sim)
DI	Safe	67.71 ± 2.95	76.04 ± 3.90	72.9 ± 8.20	$\mathbf{100.00 \pm 0.00}$	98.96 ± 1.47	$\mathbf{100.00 \pm 0.00}$
	Reach	67.71 ± 2.95	26.04 ± 7.80	94.8 ± 1.50	$\mathbf{97.92 \pm 3.61}$	96.86 ± 2.55	$\mathbf{97.92 \pm 3.61}$
	Success	67.71 ± 2.95	21.88 ± 5.10	68.8 ± 6.80	$\mathbf{97.92 \pm 3.61}$	95.83 ± 1.47	$\mathbf{97.92 \pm 3.61}$
	Time (μs)	9.24 ± 0.36	$-^{\dagger}$	6.63 ± 0.04	$\mathbf{4.99 \pm 0.02}$	8256 ± 900	$\mathbf{4.96 \pm 0.03}$
DC	Safe	56.25 ± 5.10	70.83 ± 3.90	64.6 ± 13.10	$\mathbf{97.92 \pm 1.80}$	$\mathbf{97.92 \pm 1.47}$	$\mathbf{97.92 \pm 1.80}$
	Reach	52.08 ± 3.90	2.08 ± 2.95	27.1 ± 3.90	$\mathbf{95.83 \pm 1.80}$	$\mathbf{95.83 \pm 1.47}$	94.79 ± 3.61
	Success	52.08 ± 3.90	2.08 ± 2.95	21.9 ± 2.60	$\mathbf{93.75 \pm 0.00}$	$\mathbf{94.79 \pm 1.47}$	$\mathbf{94.79 \pm 3.61}$
	Time (μs)	9.90 ± 0.21	$-^{\dagger}$	6.70 ± 0.02	$\mathbf{4.79 \pm 0.01}$	9280 ± 1262	$\mathbf{4.82 \pm 0.04}$
CF	Safe	88.54 ± 2.95	$\mathbf{98.96 \pm 1.80}$	91.7 ± 5.90	94.79 ± 3.61	93.75 ± 0.00	$\mathbf{95.83 \pm 4.77}$
	Reach	85.42 ± 3.90	0.00 ± 0.00	$\mathbf{96.9 \pm 2.60}$	94.79 ± 6.51	$\mathbf{100.00 \pm 0.00}$	93.75 ± 6.25
	Success	85.42 ± 3.90	0.00 ± 0.00	88.5 ± 3.90	$\mathbf{89.58 \pm 10.05}$	$\mathbf{93.75 \pm 0.00}$	89.58 ± 10.05
	Time (μs)	9.51 ± 0.38	$-^{\dagger}$	8.63 ± 0.03	$\mathbf{5.12 \pm 0.04}$	2773 ± 103	$\mathbf{5.08 \pm 0.03}$

We evaluate two variants of BarrierFormer that differ in how predictive rollouts are generated during training. In **Ours (Sim)**, the barrier critic h_ϕ is trained using rollouts generated by the true simulator, providing accurate state predictions for CBF constraint evaluation. In **Ours (Dyn-Head)**, the barrier critic is trained using rollouts generated by the learned dynamics head d_{θ_x}, which is jointly trained with the transformer backbone. Table 1 compares BarrierFormer against baselines across three dynamical systems, evaluated over 32 environments and 3 seeds for 256 timesteps on the training distribution. Among model-free methods, **Ours (Dyn-Head)** achieves the best balance between safety and goal-reaching with competitive inference time. CoBL-Diffusion achieves higher safety rates on CF but at the cost of near-zero reaching rates, consistent with the local trap problem identified in [18], where CBF-guided gradient projection during denoising can prevent the planner from making progress toward the goal. The higher performance on Crazyflie across all methods is attributed to the lower obstacle density in the 3D environment. Among model-based methods, **Ours (Sim)** matches or exceeds MPC's safety rate across all environments while reducing per-step computation time by two to three orders of magnitude.

Generalization Evaluation. Fig. 2 evaluates zero-shot generalization across different obstacle densities and workspace sizes. Obstacle density is defined as $\rho = M/l^{n_p}$, where M is the number of obstacles and $n_p \in \{2, 3\}$ is the dimension of the position space. We evaluate $\rho \in \{0.50, 0.75, 1.00\}$ with workspace sizes $l \in \{4, 6\}$ for DI and DC and $l \in \{3, 4\}$ for CF. The DI and DC models are trained at $\rho = 0.50$ with $l = 4$, while the CF model is trained with $M = 6$ obstacles and $l = 3$, corresponding to $\rho \approx 0.22$. To ensure a controlled, paired comparison, each instance is initialized at the highest density, $\rho = 1.00$. Lower-density variants are then obtained by removing a random subset of obstacles while preserving the poses of all remaining obstacles. Safety rates generally decrease with increasing density, as higher clutter reduces navigable space and increases the frequency of obstacle encounters. At a fixed obstacle density, performance generally decreases as the workspace size increases, potentially due to longer navigation paths and increased exposure to obstacles. Despite being trained only at $(\rho, l) = (0.50, 4)$ for DI and DC and $(\rho, l) \approx (0.22, 3)$ for CF, both variants retain substantial safety under the evaluated distribution shifts.

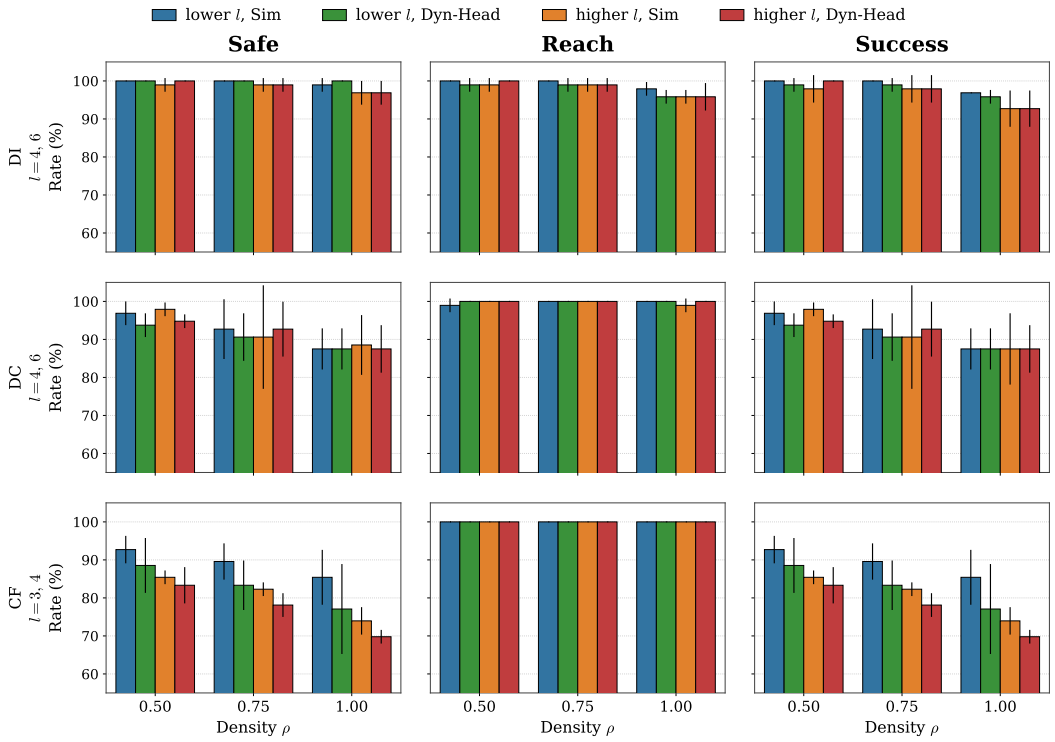


Figure 2: Zero-shot generalization across obstacle densities $\rho = M/l^{n_p}$ and workspace sizes. Rows show DI, DC, and CF; columns show safety, reaching, and success rates. Colors distinguish Sim and Dyn-Head at the smaller and larger workspace sizes. Bars report mean $\pm$ standard deviation over 32 environments and 3 seeds. The DI and DC models are trained only at $(\rho, l) = (0.50, 4)$, while the CF model is trained only at $(\rho, l) \approx (0.22, 3)$.

6 Limitations

While the proposed architecture performs better than a variety of existing prediction-based frameworks, its performance drops as problem complexity increases (from linear to nonlinear and from 2D to 3D). The resulting architecture is model-free at inference; however, the training mechanism still requires access to a black-box simulator for data collection. Making the framework truly model-free during training remains an open direction, requiring end-to-end online training to remove the simulator dependency and distributional bias introduced by offline pretraining. Beyond single-agent navigation, a natural extension is to multi-agent settings, where graph-based transformers could enable distributed horizon-level barrier enforcement across agents. Finally, similar to the existing black-box simulator-based approaches, the proposed architecture can have a large simulation-to-real-world (sim2real) gap, which might lead to undesirable behavior on real robotic systems.

7 Conclusion

In this paper, we presented BarrierFormer, a barrier-supervised transformer framework that encodes rollout-level CBF constraints directly into policy parameters via training-time supervision from an SQP-based safety teacher, enabling predictive safety enforcement without online optimization at inference. Under partial observability, BarrierFormer uses an observation–action history to generate a predictive rollout and evaluate DTCBF violations along it. Evaluations on linear and nonlinear, 2D and 3D dynamical systems illustrate that BarrierFormer achieves better performance than model-free baselines and competitive performance with model-based MPC, while reducing inference latency by over three orders of magnitude.

Acknowledgments

The Authors acknowledge the National Artificial Intelligence Research Resource (NAIRR) Pilot and AWS under grant NAIRR250034 for contributing to this research result. Any opinions, findings, conclusions, or recommendations expressed in this publication are those of the authors and do not necessarily reflect the views of the sponsors.

References

[1] O. So and C. Fan. Solving Stabilize-Avoid via Epigraph Form Optimal Control using Deep Reinforcement Learning. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.

[2] P. Palanisamy. Multi-agent connected autonomous driving using deep reinforcement learning. In *2020 International Joint Conference on Neural Networks (IJCNN)*, pages 1–7. IEEE, 2020.

[3] L. M. Schmidt, J. Brosig, A. Plinge, B. M. Eskofier, and C. Mutschler. An introduction to multi-agent reinforcement learning and review of its application to autonomous mobility. In *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, pages 1342–1349. IEEE, 2022.

[4] M. Jankovic and M. Santillo. Collision avoidance and liveness of multi-agent systems with cbf-based controllers. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 6822–6828. IEEE, 2021.

[5] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017.

[6] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431. IEEE, 2019.

[7] D. R. Agrawal and D. Panagou. Safe control synthesis via input constrained control barrier functions. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 6113–6118. IEEE, 2021.

[8] M. H. Cohen and C. Belta. Approximate optimal control for safety-critical systems with control barrier functions. In *2020 59th IEEE Conference on Decision and Control (CDC)*, pages 2062–2067, 2020.

[9] A. E. Chriat and C. Sun. On the optimality, stability, and feasibility of control barrier functions: An adaptive learning-based approach. *IEEE Robotics and Automation Letters*, 8(11):7865–7872, 2023.

[10] S. Yang, S. Chen, V. M. Preciado, and R. Mangharam. Differentiable safe controller design through control barrier functions. *IEEE Control Systems Letters*, 7:1207–1212, 2022.

[11] J. Zeng, B. Zhang, and K. Sreenath. Safety-critical model predictive control with discrete-time control barrier function. In *2021 American Control Conference (ACC)*, pages 3882–3889. IEEE, 2021.

[12] J. Liu, J. Yang, J. Mao, T. Zhu, Q. Xie, Y. Li, X. Wang, and S. Li. Flexible active safety motion control for robotic obstacle avoidance: A cbf-guided mpc approach. *IEEE Robotics and Automation Letters*, 10(3):2686–2693, 2025.

[13] M. N. Zeilinger, C. N. Jones, and M. Morari. Real-time suboptimal model predictive control using a combination of explicit mpc and online optimization. *IEEE transactions on automatic control*, 56(7):1524–1534, 2011.

[14] A. Alessio and A. Bemporad. A survey on explicit model predictive control. In *Nonlinear model predictive control: towards new challenging applications*, pages 345–369. Springer, 2009.

[15] T. Kim and D. Panagou. Visibility-aware rrt* for safety-critical navigation of perception-limited robots in unknown environments. *IEEE Robotics and Automation Letters*, 2025.

[16] M. Harms, M. Kulkarni, N. Khedekar, M. Jacquet, and K. Alexis. Neural control barrier functions for safe navigation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10415–10422. IEEE, 2024.

[17] S. Keyumarsi, M. W. S. Atman, and A. Gusrialdi. Lidar-based online control barrier function synthesis for safe navigation in unknown environments. *IEEE Robotics and Automation Letters*, 9(2):1043–1050, 2023.

[18] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus. Safediffuser: Safe planning with diffusion probabilistic models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=ig2wk7kK9J.

[19] K. Mizuta and K. Leung. Cobl-diffusion: Diffusion-based conditional robot planning in dynamic environments using control barrier and lyapunov functions. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13801–13808, 2024.

[20] R. Bonatti, S. Vemprala, S. Ma, F. Frujeri, S. Chen, and A. Kapoor. Pact: Perception-action causal transformer for autoregressive robotics pre-training. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3621–3627. IEEE, 2023.

[21] Y. Meng, S. Vemprala, R. Bonatti, C. Fan, and A. Kapoor. Conbat: Control barrier transformer for safe robot learning from demonstrations. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12857–12864. IEEE, 2024.

[22] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.

[23] W. Xiao, T.-H. Wang, R. Hasani, M. Chahine, A. Amini, X. Li, and D. Rus. Barriernet: Differentiable control barrier functions for learning of safe robot control. *IEEE Transactions on Robotics*, 39(3):2289–2307, 2023.

[24] O. So, Z. Serlin, M. Mann, J. Gonzales, K. Rutledge, N. Roy, and C. Fan. How to train your neural control barrier function: Learning safety filters for complex input-constrained systems. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11532–11539. IEEE, 2024.

[25] L. Yang, B. Werner, M. de Sa, and A. D. Ames. Cbf-rl: Safety filtering reinforcement learning in training with control barrier functions. *arXiv preprint arXiv:2510.14959*, 2025.

[26] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 3387–3395, 2019.

[27] Y. Emam, G. Notomista, P. Glotfelter, Z. Kira, and M. Egerstedt. Safe reinforcement learning using robust control barrier functions. *IEEE Robotics and Automation Letters*, 10(3):2886–2893, 2022.

[28] D. Celestini, D. Gammelli, T. Guffanti, S. D'Amico, E. Capello, and M. Pavone. Transformer-based model predictive control: Trajectory optimization via sequence modeling. *IEEE Robotics and Automation Letters*, 9(11):9820–9827, 2024.

[29] F. Blanchini. Set invariance in control. *Automatica*, 35(11):1747–1767, 1999.

[30] S. Prajna and A. Jadbabaie. Safety verification of hybrid systems using barrier certificates. In *International workshop on hybrid systems: Computation and control*, pages 477–492. Springer, 2004.

[31] A. Agrawal and K. Sreenath. Discrete control barrier functions for safety-critical control of discrete systems with application to bipedal robot navigation. In *Robotics: Science and Systems*, volume 13, pages 1–10. Cambridge, MA, USA, 2017.

[32] J. Breeden and D. Panagou. Predictive control barrier functions for online safety critical control. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 924–931. IEEE, 2022.

[33] A. Katriniok, E. Shakhesi, and W. Heemels. Discrete-time control barrier functions for guaranteed recursive feasibility in nonlinear mpc: An application to lane merging. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, pages 3776–3783. IEEE, 2023.

[34] J. Huang, H. Wang, K. Margellos, and P. Goulart. Predictive control barrier functions: Bridging model predictive control and control barrier functions. In *2025 European Control Conference (ECC)*, pages 2623–2629. IEEE, 2025.

[35] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[36] P. T. Boggs and J. W. Tolle. Sequential quadratic programming. *Acta Numerica*, 4:1–51, 1995. doi:10.1017/S0962492900002518.

[37] K.-C. Hsu*, V. Rubies-Royo*, C. Tomlin, and J. Fisac. Safety and liveness guarantees through reach-avoid reinforcement learning. In *Robotics: Science and Systems XVII*, RSS2021. Robotics: Science and Systems Foundation, 2021. URL http://dx.doi.org/10.15607/RSS.2021.XVII.077.

[38] J. F. Fisac, N. F. Lugovoy, V. Rubies-Royo, S. Ghosh, and C. J. Tomlin. Bridging hamilton-jacobi safety analysis and reinforcement learning. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8550–8556, 2019.

[39] S. Zhang, O. So, K. Garg, and C. Fan. GCBF+: A neural graph control barrier function framework for distributed safe multi-agent control. *IEEE Transactions on Robotics*, 2025.

[40] A. Sathya, P. Sopasakis, R. Van Parys, A. Themelis, G. Pipeleers, and P. Patrinos. Embedded nonlinear model predictive control for obstacle avoidance using panoc. In *2018 European control conference (ECC)*, pages 1523–1528. IEEE, 2018.

[41] Z. Wang, M. Li, A. Mandlekar, Z. Xu, J. Fan, Y. Narang, L. Fan, Y. Zhu, Y. Balaji, M. Zhou, et al. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. In *International Conference on Machine Learning*, pages 63399–63416. PMLR, 2025.

[42] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al. Jax: composable transformations of python+ numpy programs. 2018.

[43] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[44] C. Budaciu, N. Botezatu, M. Kloetzer, and A. Burlacu. On the evaluation of the crazyflie modular quadcopter system. In *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1189–1195, 2019.

[45] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer. Hg-dagger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083. IEEE, 2019.

[46] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Appendix

A Implementation and Experiment Details

We implement the framework in JAX [42] with the Flax neural network library, training all models end-to-end with the AdamW optimizer [43]. The same network architecture, optimizer, and loss formulation are used across all three environments (Double Integrator, Dubins Car, and Crazyflie); only the input and output dimensions of the observation, state, and action spaces change (refer Table 2). The policy follows a learned receding-horizon design: a causal transformer encodes the robot's recent observation-action history, a residual actor head corrects a nominal controller, and a learned dynamics head provides a differentiable one-step predictor used to roll the policy forward during training. Safety is enforced by a neural barrier critic supervised by safe and unsafe labels and a discrete-time CBF condition evaluated along transformer-predicted rollouts. An SQP-based safety teacher provides imitation targets for the actor head. All weights use Xavier-uniform initialization and all environments are integrated with a fixed timestep $\Delta t = 0.03\,\mathrm{s}$. All policies are trained for 1000 steps, with 256 timesteps of environment interaction per step.

A.1 Observation Representation

For each environment, the robot observation $\mathbf{o}_t \in \mathbb{R}^{n_o}$ concatenates the robot state $\mathbf{x}_t \in \mathbb{R}^{n_x}$, the relative goal vector $\mathbf{p}_{\text{goal}} - \mathbf{p}_t \in \mathbb{R}^{n_p}$, and the flattened LiDAR-ray features $\mathbf{y}_{t,j}$. The 2D environments (Double Integrator and Dubins Car) use four features per LiDAR ray: a hit indicator, a normalized distance, and the ray-direction components $(\cos\theta_j, \sin\theta_j)$. The 3D Crazyflie environment uses five features per LiDAR ray: a hit indicator, a normalized distance, and the ray-direction components $(\cos\theta_j \cos\phi_j, \cos\theta_j \sin\phi_j, \sin\theta_j)$. In 2D, θ_j denotes the planar angle of the j-th ray. In 3D, θ_j denotes its elevation from the horizontal plane, while ϕ_j denotes its azimuth in the horizontal plane measured from the positive x-axis. The resulting observation dimensions for each environment are summarized in Table 3.

A.2 Causal Transformer Backbone

The history encoder is a GPT-2-style [35] decoder-only transformer with pre-LayerNorm (LN) residual blocks. The transformer backbone takes the observation-action history $\mathcal{H}_t$ as input and produces a latent summary $\mathbf{z}_t \in \mathbb{R}^{d_h}$ used by the downstream actor and dynamics heads. The history contains $T = 12$ observations and $T-1$ actions, tokenized into an interleaved sequence of length $2T - 1 = 23$:

$$\big[\, \mathbf{o}_{t-T+1},\, \mathbf{u}_{t-T+1},\, \ldots,\, \mathbf{o}_{t-1},\, \mathbf{u}_{t-1},\, \mathbf{o}_t \,\big].$$

Each observation and action token is linearly projected to the model width d_h by two separate embedding layers, and a learned absolute positional embedding is added. The sequence is processed by a single GPT-2-style [35] transformer block with 2 attention heads under a causal attention mask, consisting of pre-LayerNorm multi-head self-attention (MHSA) followed by a position-wise feed-forward network with GELU activation, both with residual connections. After the final LayerNorm, the latent at the last token position (corresponding to the current observation $\mathbf{o}_t$) is extracted as $\mathbf{z}_t$. The full configuration is summarized in Table 3.

A.3 Actor Head

The actor head π_{θ_u} maps the transformer latent representation $\mathbf{z}_t \in \mathbb{R}^{d_h}$ to a residual correction $\Delta\mathbf{u}_t = \pi_{\theta_u}(\mathbf{z}_t) \in \mathbb{R}^{n_u}$ that is added to the action produced by the nominal control policy μ_{nom}: $\mathbf{u}_t = \mu_{\text{nom}}(\mathbf{x}_t) + \Delta\mathbf{u}_t$. The actor head applies an input LayerNorm, four hidden layers of width 128 with $\tanh$ activations, and a linear output layer; the environment-specific layer dimensions are summarized in Table 5. The residual correction is left unbounded by the network, and the resulting action is clipped to the environment action bounds at execution. The nominal control policy μ_{nom} is an LQR controller for the Double Integrator, a PID controller for the Dubins Car, and a two-level

Table 2: Per-environment input/output dimensions. The network architecture is identical across environments; only these dimensions change. "Feat./ray" is the number of LiDAR features per ray, so $n_o = n_x + n_p + n_{\text{rays}} \cdot (\text{Feat./ray})$.

Environment	State n_x	Action n_u	LiDAR rays n_{rays}	Feat./ray	Observation n_o
Double Integrator	4	2	32	4	134
Dubins Car	4	2	32	4	134
Crazyflie	12	4	32	5	175

Table 3: Causal transformer backbone hyperparameters (shared across environments).

Hyperparameter	**Value**
Model width d_h	128
Layers / heads	1 / 2
Head dimension d_{head}	64
FFN width	$4 \times d_h = 512$
FFN activation	GELU
Positional embedding	Learned, absolute
History length T	12
Token sequence length	$2T - 1 = 23$

LQR architecture for the Crazyflie drone, where the high-level controller outputs reference velocity and yaw-rate commands that are tracked by a low-level LQR controller.

A.4 Dynamics Head

The dynamics head predicts a state increment $\Delta\hat{\mathbf{x}}_t = d_{\theta_x}(\mathbf{z}_t, \mathbf{u}_t) \in \mathbb{R}^{n_x}$ such that $\hat{\mathbf{x}}_{t+1} = \mathbf{x}_t + \Delta\hat{\mathbf{x}}_t$. It takes the concatenated vector $[\mathbf{z}_t^\top, \mathbf{u}_t^\top]^\top \in \mathbb{R}^{d_h+n_u}$, applies an input LayerNorm followed by three hidden layers with GELU activations and a linear output layer. The environment-specific layer dimensions are summarized in Table 5. The dynamics-head parameters θ_x are optimized solely through the dynamics loss L_{dyn}. When the dynamics head is used to generate predictive rollouts for the barrier loss L_{bar}, θ_x is held fixed, while gradients with respect to its inputs $\mathbf{z}_t$ and $\mathbf{u}_t$ are retained.

A.5 Barrier Critic Architecture

The barrier critic $h_\phi : \mathbb{R}^{n_o} \to (-1, 1)$ is a feed-forward network with an input LayerNorm, hidden layers of widths $(256, 256, 128)$ with ReLU activations, and a scalar output layer followed by a $\tanh$ activation. Since the SQP safety teacher generates supervision labels by evaluating barrier constraints along predicted rollouts, using the concurrently updated critic could produce inconsistent labels across inner training epochs. Therefore, a target critic $h_{\bar{\phi}}$ is maintained using the Polyak update $\bar{\phi} \leftarrow \tau_{\text{tgt}}\phi + (1 - \tau_{\text{tgt}})\bar{\phi}$, where $\tau_{\text{tgt}} = 0.5$, and the SQP teacher uses $h_{\bar{\phi}}$ instead of h_ϕ during label generation. This keeps the supervision targets fixed across the corresponding inner training epochs while the current critic h_ϕ is updated through the barrier loss L_{bar}. The learning rates and loss coefficients for each environment are provided in Table 4.

A.6 Environment Dynamics

Double Integrator. The state of the agent is given by $\mathbf{x} = [p_x, p_y, v_x, v_y]^\top$, where $[p_x, p_y]^\top$ is the position and $[v_x, v_y]^\top$ is the velocity. The control input is $\mathbf{u} = [a_x, a_y]^\top$, representing acceleration. The continuous-time dynamics are:

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u}, \tag{18}$$

Table 4: Training and rollout hyperparameters shared across DI, DC, and CF.

Hyperparameter	Value
Actor/transformer learning rate	3×10^{-5}
Barrier-critic learning rate	3×10^{-6}
λ_{act}	0.5
λ_{bar}	0.5
λ_{dyn}	1.0
λ_{safe}	1.0
λ_{unsafe}	1.5
γ (class-$\mathcal{K}$ rate)	0.3
η_s (SQP safety margin)	0.02
η_v (rollout-loss margin)	0.04
β (log-sum-exp temperature)	20.0
η (classification margin)	0.02
History length T	12
Prediction/SQP horizon H	6
τ_{tgt}	0.5
T_{label}	32

Table 5: Prediction head layer dimensions ($d_h = 128$). Hidden activations are $\tanh$ for the actor head and GELU for the dynamics head; output layers are linear.

Environment	Actor head ($d_h \to n_u$)	Dynamics head($d_h + n_u \to n_x$)
Double Integrator	$128 \to 128 \to 128 \to 128 \to 128 \to 2$	$130 \to 128 \to 128 \to 128 \to 4$
Dubins Car	$128 \to 128 \to 128 \to 128 \to 128 \to 2$	$130 \to 128 \to 128 \to 128 \to 4$
Crazyflie	$128 \to 128 \to 128 \to 128 \to 128 \to 4$	$132 \to 128 \to 128 \to 128 \to 12$

discretized with timestep $\Delta t = 0.03$s. The control input is boundedcomponentwise as $\mathbf{u} \in [-1, 1]^2$.

Dubins Car. The state of the agent is given by $\mathbf{x} = [p_x, p_y, \theta, v]^\top$, where $[p_x, p_y]^\top$ is the position, θ is the heading angle, and v is the speed. The control input is $\mathbf{u} = [\omega, a]^\top$, containing angular velocity and longitudinal acceleration. The continuous-time dynamics are:

$$\dot{\mathbf{x}} = \begin{bmatrix} v\cos(\theta) \\ v\sin(\theta) \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u}, \tag{19}$$

discretized with timestep $\Delta t = 0.03\,\text{s}$. The control input is bounded componentwise as $\mathbf{u} \in [-3, 3]^2$. Since the heading angle θ is unbounded, the agent-state part of the Dubins Car observation uses the bounded encoding $[p_x,\ p_y,\ v\cos\theta,\ v\sin\theta]^\top$ in place of $\mathbf{x}$.

Crazyflie Drone. The Crazyflie quadrotor is modeled using 6-DOF rigid body dynamics [44] with state $\mathbf{x} \in \mathbb{R}^{12}$ consisting of positions, velocities, Euler angles, and angular velocities, and control input $\mathbf{u} = [U_1, U_2, U_3, U_4]^\top \in \mathbb{R}^4$ consisting of thrust U_1 and moments U_2, U_3, U_4. The translational dynamics are:

$$\dot{p}_x = \big(c(\phi)c(\psi)s(\theta) + s(\phi)s(\psi)\big)w - \big(s(\psi)c(\phi) - c(\psi)s(\phi)s(\theta)\big)v + uc(\psi)c(\theta) \tag{20a}$$

$$\dot{p}_y = \big(s(\phi)s(\psi)s(\theta) + c(\phi)c(\psi)\big)v - \big(c(\psi)s(\phi) - s(\psi)c(\phi)s(\theta)\big)w + us(\psi)c(\theta) \tag{20b}$$

$$\dot{p}_z = wc(\phi)c(\theta) - us(\theta) + vs(\phi)c(\theta) \tag{20c}$$

$$\dot{u} = rv - qw + gs(\theta) \tag{20d}$$

$$\dot{v} = pw - ry - gs(\phi)c(\theta) \tag{20e}$$

$$\dot{w} = qu - pv - gc(\theta)c(\phi) + \frac{U_1}{m}, \tag{20f}$$

and the rotational dynamics are:

$$\dot{\psi} = r\frac{c(\phi)}{c(\theta)} + q\frac{s(\phi)}{c(\theta)} \tag{21a}$$

$$\dot{\theta} = qc(\phi) - rs(\phi) \tag{21b}$$

$$\dot{\phi} = p + rc(\phi)t(\theta) + qs(\phi)t(\theta) \tag{21c}$$

$$\dot{p} = \frac{1}{I_{xx}}\big(U_4 - qr(I_{zz} - I_{yy})\big) \tag{21d}$$

$$\dot{q} = \frac{1}{I_{yy}}\big(U_3 - pr(I_{xx} - I_{zz})\big) \tag{21e}$$

$$\dot{r} = \frac{1}{I_{zz}}\big(U_2 - pq(I_{yy} - I_{xx})\big), \tag{21f}$$

where $c(\cdot)$, $s(\cdot)$, and $t(\cdot)$ denote $\cos(\cdot)$, $\sin(\cdot)$, and $\tan(\cdot)$, respectively; $\mathbf{p} = [p_x, p_y, p_z]^\top$ is the position, (ϕ, θ, ψ) are the Euler angles, and $g = 9.81\mathrm{m/s}^2$ is gravitational acceleration. The motor thrust-and-moment mapping is:

$$\begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} C_T & C_T & C_T & C_T \\ -dC_T\sqrt{2} & -dC_T\sqrt{2} & dC_T\sqrt{2} & dC_T\sqrt{2} \\ -dC_T\sqrt{2} & dC_T\sqrt{2} & dC_T\sqrt{2} & -dC_T\sqrt{2} \\ -C_D & C_D & -C_D & C_D \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}, \tag{22}$$

where ω_i is the angular speed of the i-th motor. System parameters are: $I_{xx} = I_{yy} = 1.395 \times 10^{-5}$ kg·m^2, $I_{zz} = 2.173 \times 10^{-5}$ kg·m^2, $m = 0.0299$ kg, $C_T = 3.1582 \times 10^{-10}$ N/rpm^2, $C_D = 7.9379 \times 10^{-12}$ N/rpm^2, and $d = 0.03973$ m [44]. The dynamics are discretized with timestep $\Delta t = 0.03$s. The policy outputs a normalized high-level command in $[-1, 1]^4$, which after scaling gives world-frame velocity targets $v_x, v_y \in [-2, 2]$ m/s, and $v_z \in [-0.5, 0.5]$ m/s.

A.7 Handcrafted CBF Definitions

For the CBF-RL and CoBL-Diffusion baselines, we use handcrafted CBFs defined based on the agent's state and obstacle positions. All CBFs are designed to certify a minimum separation distance of r between the agent and each obstacle.

Double Integrator. For a second-order system, we use a relative-degree-2 CBF. Let $h_0(\mathbf{x}) = \|\mathbf{p} - \mathbf{p}_{\text{obs}}\|^2 - 4r^2$ denote the position-based barrier, where $\mathbf{p}$ and $\mathbf{p}_{\text{obs}}$ are the agent and obstacle positions, respectively. The CBF is defined as:

$$h_1(\mathbf{x}) = \dot{h}_0(\mathbf{x}) + \alpha_1 h_0(\mathbf{x}) = 2(\mathbf{p} - \mathbf{p}_{\text{obs}})^\top(\mathbf{v} - \mathbf{v}_{\text{obs}}) + \alpha_1 h_0(\mathbf{x}), \tag{23}$$

where $\alpha_1 = 10.0$ and $\mathbf{v}$, $\mathbf{v}_{\text{obs}}$ are the agent and obstacle velocities, respectively.

Dubins Car. Similarly, a relative-degree-2 CBF is used. Let $h_0(\mathbf{x}) = \|\mathbf{p} - \mathbf{p}_{\text{obs}}\|^2 - 4r^2$. The CBF is:

$$h_1(\mathbf{x}) = 2(\mathbf{p} - \mathbf{p}_{\text{obs}})^\top(\mathbf{v} - \mathbf{v}_{\text{obs}}) + \alpha_1 h_0(\mathbf{x}), \tag{24}$$

where $\mathbf{v} = [v\cos\theta, v\sin\theta]^\top$ is the agent velocity in Cartesian coordinates and $\alpha_1 = 5.0$.

Crazyflie. Due to the high relative degree of the 6-DOF quadrotor dynamics, a relative-degree-3 CBF is required. Starting from $h_0(\mathbf{x}) = \|\mathbf{p} - \mathbf{p}_{\text{obs}}\|^2 - 4r^2$, we define:

$$h_1(\mathbf{x}) = \dot{h}_0(\mathbf{x}) + \alpha_1 h_0(\mathbf{x}), \tag{25}$$

$$h_2(\mathbf{x}) = \dot{h}_1(\mathbf{x}) + \alpha_2 h_1(\mathbf{x}), \tag{26}$$

where $\alpha_1 = 30.0$ and $\alpha_2 = 50.0$. The time derivatives are computed analytically through the Crazyflie dynamics model. The constraint $h_2(\mathbf{x}) \geq 0$ is enforced at each control step.

Table 6: Ablation study on predictive horizon H for Double Integrator with history length $T = 12$. Mean $\pm$ std over 32 environments and 3 seeds.

Method	H	Safe	Reach	Success
Ours (Dyn-Head)	1	84.38 ± 3.12	86.46 ± 7.22	76.04 ± 7.86
	3	91.67 ± 1.80	91.67 ± 3.61	78.12 ± 11.27
	6	100.00 ± 0.00	97.92 ± 3.61	97.92 ± 3.61
	9	100.00 ± 0.00	94.79 ± 3.61	94.79 ± 3.61
	12	100.00 ± 0.00	96.88 ± 3.12	96.88 ± 3.12

Table 7: Dynamics-head rollout RMSE on the Double Integrator with history length $T = 12$. The $H = 1$ column reports the one-step RMSE; the remaining columns report the autoregressive rollout RMSE over horizons H.

Input signal	$H = 1$	$H = 8$	$H = 16$	$H = 24$	$H = 32$
GCBF+	0.000282	0.00162	0.00210	0.00258	0.00310
QP	0.000913	0.00215	0.00255	0.00320	0.00393
LQR	0.000383	0.00143	0.00186	0.00253	0.00301
Actor head (BarrierFormer)	0.000350	0.00160	0.00214	0.00247	0.00301
Actor head + noise	0.000343	0.00159	0.00219	0.00271	0.00300
Constant	0.000567	0.00409	0.00395	0.00483	0.00539
Uniform random	0.001910	0.00431	0.00514	0.00507	0.00489
Matched random	0.001441	0.00449	0.00543	0.00558	0.00680
Polynomial	0.000652	0.00460	0.00499	0.00484	0.00538
Sinusoidal	0.001209	0.00365	0.00530	0.00669	0.00831
Zero	0.000204	0.00447	0.00667	0.00881	0.01113

B Evaluation

Success criteria. All metrics are computed per episode using the simulator's geometric state with agent radius $r = 0.05$. An episode is considered unsafe if the agent overlaps any obstacle at any time step, i.e., $\text{Unsafe} := \max_t \text{unsafe}_t$. An episode is considered to have reached the goal if $|\mathbf{p}_t - \mathbf{p}_{\text{goal}}| < 2r$ at any time step, i.e., $\text{Reached} := \max_t \text{finish}_t$. The three reported metrics are:

$$\text{Safety} := 1 - \text{Unsafe}, \quad \text{Reaching} := \text{Reached}, \quad \text{Success} := (1 - \text{Unsafe}) \cdot \text{Reached}. \tag{27}$$

Success requires the agent to reach the goal without any collision along the trajectory.

Aggregation. For each environment, we run $S = 3$ random seeds, evaluating each seed on $N = 32$ independently generated environments with distinct start, goal, and obstacle layouts, yielding $S \times N = 96$ episodes in total. Each seed yields one mean over its 32 episodes, and we report the mean and standard deviation of those 3 per-seed means.

Horizon Selection Table 6 studies the effect of the predictive horizon H on the Double Integrator environment using the history length $T = 12$. The $H = 1$ setting corresponds to pointwise barrier supervision without multi-step predictive rollout, and gives the lowest safety rate. Increasing the horizon from $H = 1$ to $H = 6$ improves safety from 84.38% to 100% and success from 76.04% to 97.92%, demonstrating the benefit of enforcing barrier constraints over predicted rollouts rather than only at the next step. Beyond $H = 6$, safety remains at 100%, while reaching and success rates vary within one standard deviation, so increasing H further yields no additional benefit. At the same time, longer horizons rely on longer autoregressive dynamics-head rollouts in the barrier rollout loss, whose prediction error grows with the horizon (Table 7), and introduce more linearized barrier constraints in the SQP subproblem, making the resulting correction labels more sensitive to linearization error and slack relaxation. We therefore use $H = 6$, the smallest evaluated horizon at which safety saturates, in the main experiments with $T = 12$.

Table 8: Dynamics-head rollout RMSE on the Double Integrator with history length $T = 32$. The $H = 1$ column reports the one-step RMSE; the remaining columns report the autoregressive rollout RMSE over horizons H.

Input signal	$H = 1$	$H = 8$	$H = 16$	$H = 24$	$H = 32$
GCBF+	0.000170	0.00084	0.00171	0.00246	0.00326
QP	0.000290	0.00169	0.00229	0.00310	0.00386
LQR	0.000130	0.00110	0.00176	0.00252	0.00325
Actor head (BarrierFormer)	0.000300	0.00089	0.00163	0.00243	0.00332
Actor head + noise	0.000300	0.00098	0.00164	0.00242	0.00335
Constant	0.000220	0.00176	0.00226	0.00285	0.00342
Uniform random	0.000620	0.00194	0.00217	0.00251	0.00292
Matched random	0.000540	0.00167	0.00231	0.00259	0.00322
Polynomial	0.000370	0.00203	0.00232	0.00261	0.00297
Sinusoidal	0.000360	0.00161	0.00248	0.00321	0.00410
Zero	0.000150	0.00484	0.00776	0.00961	0.01154

Pretraining. In Phase 1, we jointly pretrain the transformer backbone, actor head, and dynamics head using trajectories collected from three different control policies: GCBF+ [39], a QP safety controller, and the nominal LQR controller. Together, these policies provide a diverse training distribution comprising learned safe actions, optimization-based safety corrections, and nominal control inputs. This diversity mitigates overfitting to the narrow action distribution induced by a single controller and helps preserve rollout accuracy as predictive-horizon CBF supervision guides actor updates during Phase 2. For each transition, the executed action and observed state increment provide supervision for the dynamics head. The GCBF+ and QP trajectories also provide residual-action labels for the actor head, while the nominal LQR trajectories provide additional dynamics supervision around the nominal controller. The models with history lengths $T = 12$ and $T = 32$ use the same training data and optimization procedure.

We evaluate each pretrained dynamics head on 50 independently generated episodes of 256 steps for each test input. The closed-loop inputs comprise the nominal LQR controller, the BarrierFormer actor added to the nominal action, its noise-perturbed variant with additive Gaussian noise of standard deviation 0.1, GCBF+, and the QP controller. The open-loop inputs are generated independently for each action dimension, with their parameters resampled for every episode. Constant inputs hold a value sampled uniformly from $[-1, 1]$ throughout the episode. Uniform-random inputs are sampled independently from $\mathcal{U}(-1, 1)$ at every step, while matched-random inputs are sampled from a zero-mean Gaussian distribution with standard deviation 0.3 and clipped to the action bounds. Sinusoidal inputs use amplitudes sampled from $[0.3, 1]$, frequencies from 0.5 to 6 cycles per episode, and phases from $[0, 2\pi]$. Polynomial inputs are random cubic signals normalized to $[-1, 1]$, and zero-action inputs remain zero throughout the episode. These test families span closed-loop, smooth low-frequency, and broadband random inputs, providing a range of temporal structures not encountered during pretraining. For each input signal, the dynamics head is applied autoregressively, and state RMSE is reported for prediction horizons $H \in \{1, 8, 16, 24, 32\}$, where $H = 1$ corresponds to one-step prediction. Tables 7 and 8 present the results for $T = 12$ and $T = 32$, respectively. Increasing the history length to $T = 32$ consistently reduces prediction error on controller-based inputs through $H = 24$. At $H = 32$, the two history lengths perform comparably on these inputs, with mean RMSEs of 0.00321 and 0.00341 for $T = 12$ and $T = 32$, respectively. The longer history provides a clearer benefit on unseen nonzero open-loop inputs, reducing their mean $H = 32$ RMSE from 0.00615 to 0.00333, a reduction of approximately 46%. The zero-action input is the exception, for which both models accumulate similar long-horizon error.

Frozen-Backbone Ablation. To evaluate the contribution of online training of the transformer backbone and dynamics head, we evaluate a variant in which the transformer backbone and dynamics head are frozen after Phase 1 pretraining, and only the actor head and barrier critic are trained during Phase 2. The results reported in Table 9 show degraded performance compared to the full

Table 9: Ablation study on freezing the transformer backbone and dynamics head after Phase 1. The frozen variant trains only the actor head and barrier critic in Phase 2, while *Ours* denotes the full training setup. Mean $\pm$ standard deviation across 3 seeds, with each seed evaluated on 32 environments.

Env.	Safe		Reach		Success	
	Frozen	Ours	Frozen	Ours	Frozen	Ours
DI	90.62 ± 5.41	$\mathbf{100.00 \pm 0.00}$	91.67 ± 4.77	$\mathbf{97.92 \pm 3.21}$	84.38 ± 8.27	$\mathbf{97.92 \pm 3.21}$
DC	82.29 ± 1.80	$\mathbf{97.92 \pm 1.80}$	$\mathbf{96.88 \pm 3.12}$	95.83 ± 1.80	80.21 ± 1.80	$\mathbf{93.75 \pm 0.00}$
CF	93.75 ± 3.12	$\mathbf{94.79 \pm 3.61}$	$\mathbf{95.79 \pm 6.51}$	94.79 ± 6.51	88.54 ± 9.02	$\mathbf{89.58 \pm 10.05}$

online training setup described in Section 4, which we attribute to two factors. First, the frozen dynamics head accumulates prediction errors relative to the true simulator as the actor head's action distribution shifts away from the pretraining distribution; since the transformer backbone is also frozen, the latent representation $\mathbf{z}_t$ cannot adapt to the shifting action distribution, resulting in inaccurate rollout predictions for barrier constraint evaluation. This follows the same motivation as the DAgger framework [45]: collecting data under the current policy reduces distribution shift, while here the resulting on-policy transitions allow the dynamics head to adapt to the policy's evolving state-action distribution. Second, freezing the backbone fixes the latent representation $\mathbf{z}_t$ to features learned in Phase 1 for dynamics prediction and behavior cloning, which are not optimized for safety. In the full setup, the actor and barrier losses in (16) also update θ_T, allowing the attention mechanism to adapt $\mathbf{z}_t$ to safety-relevant parts of the observation-action history; with a frozen backbone, barrier supervision can only reshape the mapping from a fixed feature space to actions. Together, these results motivate the full Phase 2 training, in which all components are updated jointly as described in Section 4.

C Baseline Implementation

All four baselines are re-implemented in JAX/Flax within our simulation framework using the same dynamics, observation model, obstacle configurations, and evaluation protocol. Specifically, all methods are evaluated over episodes of 256 timesteps with $\Delta t = 0.03$s, on a $l = 4$ workspace with 8 obstacles for DI and DC, and a $l = 3$ workspace with 6 obstacles for CF, across 3 seeds $\times$ 32 environments.

C.1 ConBaT

We re-implement ConBaT [21] in JAX/Flax following the original formulation. ConBaT learns a safe policy from demonstrations in two stages on top of a PACT causal transformer backbone [20], and enforces safety at deployment through a conditional gradient-based action correction governed by a learned discrete-time control barrier function.

Data. Safe trajectories are collected by rolling out an LQR reference controller, and unsafe trajectories are collected by adding Gaussian exploration noise ($\sigma = 0.5$) until a collision occurs. We collect 7,500 safe and 2,500 unsafe trajectories of length 256, sliced into context windows of length $T_{\mathrm{ConBaT}} = 16$.

Architecture. The PACT backbone processes an interleaved state-action token sequence with a GPT-2-style causal transformer (2 layers, embedding size 64, 8 heads, context $T_{\mathrm{ConBaT}} = 16$), with a policy head and a world-model head. Two barrier critics are added: a current-state critic $C(\mathbf{s}_t^+) \in \mathbb{R}$ and a future-state critic $C_f(\mathbf{s}_t^+, \mathbf{a}_t^+) \in \mathbb{R}$, each a two-layer MLP with 128 hidden units and $\tanh$ activation. By convention, $C(\mathbf{s}_t^+) > 0$ denotes safety.

Phase I: PACT pre-training. The PACT backbone is pre-trained for 50 epochs (batch 32, Adam, learning rate 10^{-4}) to minimize a masked action-prediction loss on safe timesteps and a world-model next-state-embedding loss weighted by $\lambda_{\text{wm}} = 0.1$.

Phase II: Barrier critic training. With PACT frozen, the two barrier critics are trained for 10 epochs with three losses: a classification loss pushing $C(\mathbf{s}_t^+) > \gamma_{\text{ConBaT}}$ on safe states and $C(\mathbf{s}^+t) < -\gamma_{\text{ConBaT}}$ on unsafe states, a DTCBF forward-invariance loss enforcing $C(\mathbf{s}_{t+1}^+) \geq (1 - \alpha_{\text{ConBaT}})C(\mathbf{s}^+t)$, and a consistency loss training C_f to predict the next-state safety score. We use $\gamma_{\text{ConBaT}} = 1.0$, $\alpha_{\text{ConBaT}} = 0.1$, and loss weights $(\lambda_{\text{c}}, \lambda_{\text{s}}, \lambda_{\text{f}}) = (1, 5, 1)$.

Deployment. At each step, the policy proposes an action $\hat{\mathbf{a}}_t$, whose action embedding $\hat{\mathbf{a}}_t^+$ is evaluated by the future-state critic. If $C_f(\mathbf{s}_t^+, \hat{\mathbf{a}}_t^+) < 0$, the predicted next state is classified as unsafe and a gradient-descent correction is applied ($n_{\text{corr}} = 3$ steps, step size 0.05, and regularization $\lambda_{\text{reg}} = 0.1$); otherwise, the proposed action is executed unchanged.

C.2 CBF-RL

We re-implement CBF-RL [25] in JAX/Flax following the original formulation. CBF-RL trains a PPO policy [46] under two complementary CBF-based safety mechanisms during training: a closed-form CBF-QP action filter and a barrier-inspired reward penalty.

Network architecture. The actor-critic uses a two-hidden-layer MLP with width 256 and ReLU activations. The input concatenates the robot state, relative goal vector, and flattened obstacle encoding. Per-environment dimensions are summarized in Table 10.

Table 10: CBF-RL actor-critic input and output dimensions per environment.

Environment	Input dim	Action dim	Obstacles
Double Integrator	48	2	8 rectangular
Dubins Car	48	2	8 rectangular
Crazyflie	36	4	6 spherical

PPO training. Training uses PPO with GAE ($\gamma_{\text{PPO}} = 0.99$, $\lambda_{\text{GAE}} = 0.95$, clip $\epsilon_{\text{clip}} = 0.2$), Adam optimizer (learning rate 3×10^{-4}), global-norm gradient clipping (0.5), 10 epochs and 8 mini-batches per update, and 256 parallel environments.

CBF safety filter. The handcrafted CBFs defined in Appendix A.7 are used for each environment. Writing the CBF condition as $\psi = \mathbf{a}^\top \mathbf{u} \cdot \text{des} + b$, the closed-form filtered action is:

$$\mathbf{u} = \mathbf{u}_{\text{des}} - \frac{\min(\psi, 0)}{\|\mathbf{a}\|_2^2}\mathbf{a}, \tag{28}$$

so the action is modified only when $\psi < 0$. A barrier penalty $10 \cdot \min(h_{\min}, 0)$ is added to the reward at each step. At inference, the closed-form CBF filter remains active as a runtime safety layer.

C.3 CoBL-Diffusion

We re-implement CoBL-Diffusion [19] in JAX/Flax following the original formulation. CoBL-Diffusion is a receding-horizon diffusion planner that generates a horizon-length control sequence guided at sampling time by gradient-based CBF and CLF terms.

Data. We collect $50{,}000$ expert trajectories of length 256 per environment by rolling out the reference LQR controller. Raw states are converted to $(\mathbf{u}_t, \mathbf{p}_t)$ pairs and normalized by the arena side length $l = 4$ for DI/DC and $l = 3$ for CF.

Model and training. The denoiser is a 1-D temporal U-Net (base width 32, channel multipliers $[1, 2, 4, 8]$) that predicts the clean control sequence $\mathbf{U}^{(0)}$ from a noised sequence, conditioned on the diffusion time step and a position trajectory initialized as the straight start-goal interpolation. A cosine β schedule with 1000 diffusion steps is used. Training minimizes a three-term objective:

$$L = L_{\text{simple}} + L_{\text{traj}} + 10\, L_{\text{term}}. \tag{29}$$

The objective consists of an ℓ_1 denoising loss, a trajectory-reconstruction loss obtained by integrating the denoised controls through the true dynamics, and a terminal goal-reaching loss. Training uses Adam (learning rate 2×10^{-5}, global-norm clip 5.0), batch size 512, 1500 epochs, and an exponential moving average (EMA) of the model weights with decay 0.995.

Deployment. At inference, the planner operates in a receding-horizon fashion: it generates an $H_{\text{CoBL}} = 80$-step control sequence, applies only the first control, and then replans from the resulting state. The horizon of $H_{\text{CoBL}} = 80$ steps is chosen to balance planning accuracy against the computational cost of DDIM sampling with gradient guidance at each denoising step.

C.4 Model Predictive Control (MPC)

We implement MPC [40] as a classical model-based non-learning baseline using CasADi with the IPOPT interior-point solver.

Optimal control problem. At each control step, the MPC solves an $H_{\text{MPC}} = 20$-step trajectory optimization problem (0.6, s horizon) over states $\mathbf{X} \in \mathbb{R}^{n_x \times (H_{\text{MPC}}+1)}$ and controls $\mathbf{U} \in \mathbb{R}^{n_u \times H_{\text{MPC}}}$. The initial state is pinned as an equality constraint, dynamics are enforced as Euler-step equality constraints, and state/control limits are imposed as box constraints. The cost combines a running and terminal quadratic position error toward the goal and a control-effort regularization term. The solution is warm-started at each step by shifting the previous solution, reducing IPOPT iterations from $\sim$50 to $\sim$5. Only the first control u_0 is applied at each step.

Obstacle handling. For Double Integrator and Dubins Car, rectangular obstacles are approximated by their circumscribed circles, imposing $|\mathbf{p}_k - \mathbf{c}_i|^2 \geq (r + r_i + \epsilon)^2$ at every node. This is conservative but smooth and always feasible. For Crazyflie, spherical obstacles admit an exact distance constraint. The safe/unsafe metric is always computed using the exact obstacle geometry, independent of the planner's conservative approximation.

Robustness to model perturbations. Table 11 reports the performance of BarrierFormer under dynamics perturbations not encountered during training. The policy is trained using the unperturbed Double Integrator dynamics and evaluated without retraining under two test-time perturbations. First, we introduce a parametric velocity mismatch,

$$\dot{\mathbf{p}} = \alpha_{\text{vel}} \mathbf{v},$$

where $\alpha_{\text{vel}} = 1$ corresponds to the unperturbed dynamics, and $\alpha_{\text{vel}} \in [0.8, 1.2]$ changes how fast the robot moves for the same velocity state. Second, we add state-wise scaled process noise at every control step ($\Delta t = 0.03$ s),

$$\mathbf{x}_{t+1} = F(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t, \qquad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_w), \qquad \mathbf{\Sigma}_w = \text{diag}(\sigma_p^2, \sigma_p^2, \sigma_v^2, \sigma_v^2),$$

where F denotes the discretized dynamics, and σ_p and σ_v denote the position and velocity disturbance standard deviations, respectively. In these experiments, $\sigma_p = 4\sigma$ and $\sigma_v = 0.5\sigma$, scaled by the workspace size and the velocity limit, respectively. As shown in Table 11, under velocity mismatch, the safety rate remains at 100% across all tested values of α_{vel}. The reaching rate decreases from 97.92% under the unperturbed dynamics to 80.21% at $\alpha_{\text{vel}} = 0.8$, consistent with slower motion under a fixed episode time limit. Additive process noise is more challenging: the safety rate remains at or above 97.92% for $\sigma \leq 0.0025$, but decreases to 90.62% at $\sigma = 0.005$ and 54.17% at $\sigma = 0.01$. These results are empirical robustness tests, not formal robust safety guarantees.

Table 11: Robustness on the Double Integrator environment under dynamics perturbations. Policies are trained using the unperturbed dynamics and evaluated without retraining. Rates are reported as mean $\pm$ standard deviation over 3 seeds with 32 environments per seed. For process noise, $\sigma_p = 4\sigma$ and $\sigma_v = 0.5\sigma$.

Perturbation	Value	$T = 12,\ H = 6$		
		Safe	Reach	Success
$\dot{\mathbf{p}} = \alpha_{\text{vel}}\mathbf{v}$	0.8	100.00 ± 0.00	80.21 ± 6.51	80.21 ± 6.51
	0.9	100.00 ± 0.00	92.71 ± 4.77	92.71 ± 4.77
	1.0	100.00 ± 0.00	97.92 ± 3.61	97.92 ± 3.61
	1.1	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00
	1.2	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00
$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) + \mathbf{w}$	$\sigma = 0$	100.00 ± 0.00	97.92 ± 3.61	97.92 ± 3.61
	$\sigma = 0.001$	100.00 ± 0.00	96.88 ± 5.41	96.88 ± 5.41
	$\sigma = 0.0025$	97.92 ± 3.61	96.88 ± 5.41	95.83 ± 7.22
	$\sigma = 0.005$	90.62 ± 8.27	93.75 ± 10.83	86.46 ± 15.42
	$\sigma = 0.0075$	75.00 ± 6.25	89.58 ± 10.97	69.79 ± 14.77
	$\sigma = 0.01$	54.17 ± 9.02	86.46 ± 13.01	51.04 ± 14.43

D Inference Latency Measurement

All inference latency measurements are performed on a single workstation equipped with an NVIDIA RTX 4000 Ada Generation GPU (20 GB), an Intel Xeon w5-3423 CPU (12 cores, up to 4.2 GHz), and 62 GiB of RAM, running Ubuntu 22.04. Latency is measured as the per-step wall-clock time averaged over a full rollout of 256 steps across 32 parallel environments, with a single untimed warm-up run to exclude just-in-time (JIT) compilation cost. `jax.block_until_ready()` is called after every timed step to account for asynchronous dispatch. The per-step latency reported in Table 1 is the total batched wall time divided by the number of steps.